PostgreSQL Complete Guide — Production-Grade Mastery from Zero to Staff Engineer

Table of Contents

---

1. What Is This Repository?

This is a **comprehensive, production-grade PostgreSQL learning repository** built for engineers who want to go from zero SQL knowledge to staff-level PostgreSQL mastery. It is not a cheat-sheet collection, a link dump, or a surface-level tutorial. Every chapter contains:

- **Conceptual explanations** written at the depth required for production engineering
- **Runnable SQL examples** with expected output and annotations
- **Real-world scenarios** drawn from high-traffic systems at companies like Uber, Netflix, Shopify, and GitLab
- **Common mistakes** and how to avoid them
- **Interview-grade questions** at the end of every chapter
- **Projects** that build on each other into a portfolio-worthy capstone

PostgreSQL is the world's most advanced open-source relational database. As of 2025, it is the **number-one most-used database** among professional developers (Stack Overflow Developer Survey, 2024). Mastering it opens doors to backend engineering, data engineering, analytics engineering, and database administration roles at top-tier companies.

This guide takes you through every layer of PostgreSQL — from `SELECT` statements to WAL internals, from index design to replication topology, from row-level security to Patroni-based high-availability clusters.

---

2. Who Is This For?

Role	Benefit
Complete Beginners	Start from zero. Learn SQL, then PostgreSQL, with no assumed knowledge.
Backend Engineers	Master connection pooling, query performance, schema design, and ACID guarantees for production APIs.
Data Engineers	Deep-dive into partitioning, logical replication, FDWs, and ETL patterns within PostgreSQL.
Analytics Engineers	Window functions, CTEs, materialized views, query plan reading, and dbt integration patterns.
Database Engineers / DBAs	Full coverage of internals, VACUUM, autovacuum tuning, WAL, PITR, and HA architecture.
Interview Candidates	Structured preparation for SQL rounds at FAANG and top-tier startups. Curated question banks and walkthroughs.
Computer Science Students	Academic-quality explanations of relational theory, transaction isolation, and database internals.

---

3. What You Will Learn

By completing this repository in full, you will be able to:

SQL Foundations

- Write correct, efficient SQL across all major query patterns
- Use aggregations, subqueries, CTEs, and window functions fluently
- Model relational data with proper normalization and constraint design
- Understand and apply all four SQL sublanguages: DDL, DML, DQL, DCL

PostgreSQL Core

- Explain how PostgreSQL processes a query from parse tree to execution
- Configure PostgreSQL for development and production workloads
- Manage users, roles, schemas, and access control correctly
- Use PostgreSQL-specific features: JSONB, arrays, full-text search, custom types, extensions

Indexes and Query Optimization

- Choose the correct index type (B-Tree, Hash, GiST, GIN, BRIN, partial, expression)
- Read and interpret `EXPLAIN ANALYZE` output at expert level
- Identify and fix slow queries using `pg_stat_statements` and auto_explain
- Understand cost estimation, the planner, and statistics

Transactions and Concurrency

- Explain MVCC and how PostgreSQL implements it
- Choose the correct isolation level for each use case
- Detect and resolve deadlocks, lock contention, and long-running transactions

- Understand row-level locking, advisory locks, and `SELECT FOR UPDATE`

PostgreSQL Internals

- Explain heap file layout, page structure, and tuple visibility
- Describe how WAL works and why it enables crash recovery and replication
- Understand TOAST, bloat, VACUUM, and the visibility map
- Trace a write operation from client to WAL to data file

Production PostgreSQL

- Configure `postgresql.conf` and `pg_hba.conf` for production
- Tune autovacuum, checkpoint, BGWriter, and shared_buffers
- Set up streaming replication and logical replication
- Implement PITR and disaster recovery strategies
- Deploy PgBouncer for connection pooling
- Design a Patroni-based HA cluster

Security

- Implement row-level security (RLS) policies
- Use privilege escalation safely with `SECURITY DEFINER`
- Audit access with `pgaudit`
- Harden PostgreSQL against common attack vectors

---

4. Repository Structure

```
postgresql-complete-guide/
│
├── README.md                          ← You are here
├── LEARNING_PATH.md                   ← Role-based learning paths
├── ROADMAP.md                         ← Visual skill progression roadmap
├── GLOSSARY.md                        ← 150+ term PostgreSQL glossary
├── INTERVIEW_PREPARATION.md           ← Complete interview prep guide
│
├── 01_Fundamentals/                   ← Relational theory, SQL intro, tooling
├── 02_SQL_Basics/                     ← SELECT, WHERE, JOIN, GROUP BY
├── 03_Intermediate_SQL/               ← Subqueries, CTEs, window functions
├── 04_Advanced_SQL/                   ← Recursive CTEs, pivots, advanced patterns
├── 05_PostgreSQL_Core/                ← psql, data types, extensions, configs
├── 06_Database_Design/                ← Normalization, ERD, constraints
├── 07_Indexes/                        ← All index types, design, maintenance
├── 08_Query_Optimization/             ← EXPLAIN, planner, statistics, tuning
├── 09_Transactions_Concurrency/       ← ACID, MVCC, isolation, locking
├── 10_PostgreSQL_Internals/           ← WAL, TOAST, heap, buffers
├── 11_Advanced_PostgreSQL/            ← JSONB, FTS, arrays, custom types
├── 12_Production_PostgreSQL/          ← Config tuning, monitoring, alerting
├── 13_Replication_HA/                 ← Streaming, logical, Patroni, PgBouncer
├── 14_Security/                       ← RLS, roles, pgaudit, hardening
├── 15_Backup_Recovery/                ← pg_dump, pg_basebackup, PITR, pgBackRest
├── 16_PostgreSQL_For_Data_Engineering/ ← Partitioning, FDW, ETL, analytics
├── 17_PostgreSQL_For_Backend_Engineers/ ← Schema patterns, pooling, migrations
├── 18_Architecture_Case_Studies/      ← Real-world system design with PostgreSQL
```

```
├── 19_Interview_Questions/          ← Curated question bank by topic
├── 20_Interview_Tasks/              ← Hands-on interview-style exercises
├── 21_System_Design/                ← DB-focused system design walkthroughs
├── 22_Projects/                     ← Standalone portfolio projects
├── 23_Company_Preparation/          ← Amazon, Google, Meta, Uber, Netflix guides
├── 24_Cheat_Sheets/                 ← Quick-reference cards
└── 25_Capstone_Program/             ← End-to-end capstone challenge
```

---

5. Quick Start Guide

Option A — Complete Beginner (No SQL experience)

```bash
# 1. Clone or download this repository
git clone https://github.com/your-handle/postgresql-complete-guide.git
cd postgresql-complete-guide

# 2. Install PostgreSQL 16
# macOS
brew install postgresql@16

# Ubuntu/Debian
sudo apt install postgresql-16

# Windows
# Download from https://www.postgresql.org/download/windows/

# 3. Start PostgreSQL
pg_ctl start -D /usr/local/var/postgresql@16   # macOS
sudo systemctl start postgresql                 # Linux

# 4. Create a practice database
psql -U postgres -c "CREATE DATABASE pg_practice;"
psql -U postgres -d pg_practice

# 5. Open your first lesson
cd 01_Fundamentals
```

Option B — Already Know SQL, Learning PostgreSQL Internals

Start at `05_PostgreSQL_Core` and follow the **Backend Engineer** path in `LEARNING_PATH.md`.

Option C — Interview Prep Only (Time-Constrained)

Go directly to `LEARNING_PATH.md` → **Interview-Only Fast Track**, then work through `19_Interview_Questions` and `20_Interview_Tasks`.

Option D — Data Engineering Focus

Follow the **Data Engineer** path in `LEARNING_PATH.md`. Key chapters: `03`, `04`, `08`, `09`, `16`.

---

6. Learning Paths at a Glance

Path	Duration	Starting Point	Key Chapters
Complete Beginner	16–20 weeks	01_Fundamentals	01 → 25
Backend Engineer	8–10 weeks	05_PostgreSQL_Core	05,06,07,08,09,12,13,17
Data Engineer	8–10 weeks	03_Intermediate_SQL	03,04,08,09,16
Analytics Engineer	6–8 weeks	02_SQL_Basics	02,03,04,07,08
Database Engineer	12–14 weeks	05_PostgreSQL_Core	05–15
Interview Fast Track	3–4 weeks	19_Interview_Questions	19,20,24

See `LEARNING_PATH.md` for full details, milestones, and week-by-week plans.

---

7. How to Use This Repository

Reading Strategy

Each folder contains one or more `.md` lesson files, `.sql` exercise files, and a `README.md` that describes the folder's contents. The recommended reading order within each folder is:

1. Read the folder `README.md` to understand scope and objectives
2. Read each lesson file in numbered order
3. Run every SQL example in `psql` or your preferred client
4. Complete the exercises at the end of each lesson
5. Answer the interview questions without looking at answers first

Practice Philosophy

Do not copy-paste SQL. Type every query yourself. The muscle memory of writing SQL syntax correctly under pressure is a skill that only develops through repetition. When you make a mistake, read the error message carefully — PostgreSQL error messages are among the most informative of any database.

Spaced Repetition

For interview preparation, use the question banks in `19_Interview_Questions` with spaced repetition. Mark questions as:

- **Green** — Can answer fluently without notes
- **Yellow** — Can answer but need to think carefully
- **Red** — Cannot answer or answer is incomplete

Revisit red and yellow questions every 3 days until they are green.

Project-Based Learning

The `22_Projects` folder contains standalone projects that apply knowledge from multiple chapters. Complete at least one project per learning phase. These projects are designed to be portfolio-worthy — you can showcase them on GitHub or discuss them in interviews.

---

8. Prerequisites

Hard Prerequisites (Must Have)

- Basic programming ability in any language (Python, JavaScript, Java, etc.)
- Comfort using a terminal/command line
- Ability to install software on your computer

Soft Prerequisites (Helpful but Not Required)

- Some exposure to data in tabular form (spreadsheets count)
- Familiarity with the concept of a database (even at a high level)
- Basic Linux/macOS command-line comfort

What You Do NOT Need

- Prior SQL experience (the beginner path starts from zero)
- Prior PostgreSQL experience
- A computer science degree
- Experience with other databases

---

9. Environment Setup

Recommended Client Tools

Tool	Use Case	Platform
`psql`	Primary CLI client, always available	All
pgAdmin 4	GUI for exploring schemas and running queries	All
DBeaver	Multi-database GUI, excellent for ERD	All
DataGrip	Professional IDE for SQL (paid)	All
TablePlus	Lightweight, fast GUI	macOS/Windows

Recommended PostgreSQL Extensions (Install Early)

```
-- Performance analysis
CREATE EXTENSION pg_stat_statements;
CREATE EXTENSION auto_explain;

-- Full-text search
CREATE EXTENSION unaccent;
CREATE EXTENSION pg_trgm;

-- UUID generation
CREATE EXTENSION "uuid-ossp";
CREATE EXTENSION pgcrypto;

-- Additional data types
CREATE EXTENSION hstore;
```

```
CREATE EXTENSION citext;

-- Scheduling (optional)
CREATE EXTENSION pg_cron;
```

Docker Setup (Recommended for Isolation)

```
# docker-compose.yml — place in repo root
version: '3.8'
services:
  postgres:
    image: postgres:16
    container_name: pg_practice
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
      POSTGRES_DB: pg_practice
    ports:
      - "5432:5432"
    volumes:
      - pg_data:/var/lib/postgresql/data
      - ./init:/docker-entrypoint-initdb.d
    command: >
      postgres
        -c shared_preload_libraries='pg_stat_statements,auto_explain'
        -c pg_stat_statements.track=all
        -c auto_explain.log_min_duration=100
        -c log_min_duration_statement=100
        -c log_checkpoints=on
        -c log_lock_waits=on

volumes:
  pg_data:
```

```
# Start the environment
docker-compose up -d

# Connect
psql -h localhost -U postgres -d pg_practice

# Stop
docker-compose down
```

postgresql.conf Settings for Learning

```
# Add to postgresql.conf or pass via docker command

# Enable query statistics
shared_preload_libraries = 'pg_stat_statements,auto_explain'
```

```
pg_stat_statements.track = all

# See slow queries
log_min_duration_statement = 100    # log queries over 100ms
auto_explain.log_min_duration = 100
auto_explain.log_analyze = on
auto_explain.log_buffers = on

# See what's happening
log_checkpoints = on
log_lock_waits = on
log_temp_files = 0                  # log all temp files
deadlock_timeout = 1s

# Development-friendly settings
log_statement = 'ddl'              # log all DDL
```

---

10. Folder-by-Folder Guide

01 — Fundamentals

What: The relational model, SQL history, how databases work, client/server architecture, psql basics, pgAdmin setup. **Why:** Understanding *why* relational databases exist and how they are structured makes every subsequent topic click faster. **Outputs:** Can connect to PostgreSQL, run basic queries, understand the client/server model.

02 — SQL Basics

What: SELECT, FROM, WHERE, ORDER BY, LIMIT, OFFSET, aggregate functions (COUNT, SUM, AVG, MIN, MAX), GROUP BY, HAVING, all JOIN types (INNER, LEFT, RIGHT, FULL, CROSS, SELF), set operators (UNION, INTERSECT, EXCEPT), INSERT, UPDATE, DELETE, UPSERT. **Why:** These are the building blocks of everything else. 80% of day-to-day SQL is here. **Outputs:** Can write production-quality queries for common business questions. Passes most junior interview SQL rounds.

03 — Intermediate SQL

What: Subqueries (scalar, row, table, correlated), CTEs (`WITH` clauses), window functions (ROW_NUMBER, RANK, DENSE_RANK, LEAD, LAG, FIRST_VALUE, LAST_VALUE, SUM OVER, running totals, sliding windows, PARTITION BY, ROWS vs RANGE frames). **Why:** Window functions and CTEs are the most-tested topics in data/analytics engineering interviews. **Outputs:** Can write complex analytical queries. Passes intermediate interview SQL rounds.

04 — Advanced SQL

What: Recursive CTEs, lateral joins, pivot and unpivot patterns, conditional aggregation, gaps and islands, temporal queries, advanced date/time handling, string manipulation, regex in SQL. **Why:** These patterns appear in senior-level interviews and production analytics workloads. **Outputs:** Can solve any SQL puzzle. Ready for senior-level SQL interview rounds.

05 — PostgreSQL Core

What: PostgreSQL-specific architecture, data types (numeric, text, UUID, arrays, JSONB, hstore, ranges, geometric, network), type casting, operator overloading, schemas, search_path, extensions, `psql` meta-commands, `pg_catalog`, `information_schema`. **Why:** PostgreSQL's extended type system is a major competitive advantage. Understanding it deeply prevents schema design mistakes. **Outputs:** Can design PostgreSQL-native schemas. Knows when to use JSONB vs normalized tables.

06 — Database Design

What: Entity-Relationship Diagrams, normalization (1NF through BCNF and 3NF), denormalization patterns, primary keys (natural vs surrogate, serial vs UUID), foreign keys, constraints (NOT NULL, UNIQUE, CHECK, EXCLUDE), naming conventions, schema versioning, multi-tenancy patterns. **Why:** Bad schema design is the root cause of most slow-query and data integrity problems in production. **Outputs:** Can design a normalized, production-quality schema from requirements. Can review and critique existing schemas.

07 — Indexes

What: How B-Tree indexes work (internal structure, splits, height), Hash indexes, GIN indexes (arrays, JSONB, full-text), GiST indexes (geometric, range types), BRIN indexes (time-series data), partial indexes, expression indexes, covering indexes (INCLUDE), multi-column index column ordering, index bloat, REINDEX, index maintenance costs. **Why:** Index design is the highest-leverage optimization skill. One correct index can reduce query time from minutes to milliseconds. **Outputs:** Can design an index strategy for any schema. Can identify missing or redundant indexes. Understands the trade-off between read and write performance.

08 — Query Optimization

What: The PostgreSQL query planner, cost estimation model, statistics (`pg_statistic`, `pg_stats`), `ANALYZE`, `EXPLAIN`, `EXPLAIN ANALYZE`, `EXPLAIN (ANALYZE, BUFFERS, FORMAT JSON)`, plan nodes (Seq Scan, Index Scan, Index Only Scan, Bitmap Scan, Hash Join, Merge Join, Nested Loop), join ordering, `enable_*` planner flags, `pg_stat_statements`, `auto_explain`, identifying slow queries, fixing N+1 queries, query rewrites. **Why:** Query optimization is a core DBA and senior backend skill. `EXPLAIN ANALYZE` is the most powerful diagnostic tool in PostgreSQL. **Outputs:** Can read any query plan. Can identify bottlenecks and fix them. Ready to discuss query performance in any interview.

09 — Transactions and Concurrency

What: ACID properties, transaction syntax, savepoints, MVCC (how PostgreSQL implements multi-version concurrency control), transaction isolation levels (Read Uncommitted, Read Committed, Repeatable Read, Serializable), read phenomena (dirty read, non-repeatable read, phantom read), locking (table locks, row locks, advisory locks), `SELECT FOR UPDATE`, `SELECT FOR SHARE`, `NOWAIT`, `SKIP LOCKED`, deadlock detection and avoidance, lock monitoring (`pg_locks`, `pg_blocking_pids`). **Why:** Concurrency bugs cause data corruption and production incidents. This is the most-tested internals topic at senior levels. **Outputs:** Can explain MVCC in depth. Can diagnose and fix lock contention. Can choose isolation levels correctly.

10 — PostgreSQL Internals

What: Heap file layout (pages, tuples, ItemId array, tuple header, xmin/xmax), page structure (pd_lsn, pd_lower, pd_upper, pd_special), WAL (Write-Ahead Log) — purpose, structure, LSN, WAL writer, checkpoint, BGWriter, shared buffer pool, buffer eviction (clock-sweep), TOAST (The Oversized-Attribute Storage Technique), free space map (FSM), visibility map (VM), pg_filenode, OIDs, relation forks. **Why:** Understanding internals makes you a 10x better DBA and earns deep respect in staff-level interviews. **Outputs:** Can trace a write operation from SQL to disk. Can explain WAL's role in replication and recovery. Can discuss MVCC at the storage layer.

11 — Advanced PostgreSQL

What: JSONB operators and indexing strategies, full-text search (tsvector, tsquery, GIN indexes, ranking, phrase search), arrays and array operators, range types, composite types, domains, custom base types, custom operators, custom aggregate functions, foreign data wrappers (postgres_fdw, file_fdw), event triggers, logical decoding, publication/subscription, pg_logical. **Why:** These features make PostgreSQL a platform, not just a database. They eliminate the need for separate search, document, and messaging systems. **Outputs:** Can implement full-text search, JSONB document storage, and cross-database queries in PostgreSQL.

12 — Production PostgreSQL

What: `postgresql.conf` tuning guide (memory settings, WAL settings, query planner settings, logging), `pg_hba.conf` authentication, connection pooling concepts, PgBouncer installation and configuration (session vs transaction vs statement mode), connection limits, idle connections, monitoring stack (Prometheus + postgres_exporter + Grafana), alerting rules, `pg_activity`, `pg_top`, long-running query detection, bloat monitoring. **Why:** A PostgreSQL installation that works in development will fail in production without proper tuning. This chapter prevents production incidents. **Outputs:** Can tune a PostgreSQL instance for production workloads. Can set up monitoring and alerting. Can configure PgBouncer.

13 — Replication and High Availability

What: Physical vs logical replication, streaming replication setup (primary + replica + `pg_hba.conf` + `recovery.conf` / `postgresql.conf` standby settings), replication slots, replication lag monitoring, logical replication (publications, subscriptions, conflicts), Patroni architecture, etcd/consul/ZooKeeper for DCS, automatic failover, switchover, PgBouncer in front of Patroni, `pg_rewind`, cascading replication, hot standby queries. **Why:** HA is a mandatory requirement for any production database. Understanding Patroni is a senior/staff-level differentiator. **Outputs:** Can design and implement a Patroni-based HA cluster. Can set up streaming and logical replication. Can explain failover process in system design interviews.

14 — Security

What: Authentication methods (md5, scram-sha-256, cert, ldap, peer, ident), `pg_hba.conf` deep dive, roles and privileges (GRANT, REVOKE, privilege hierarchy), row-level security (RLS) — enabling, creating policies, USING vs WITH CHECK, `SECURITY DEFINER` vs `SECURITY INVOKER`, `pgaudit` installation and configuration, SSL/TLS for connections, secrets management, column-level encryption, privilege escalation vectors and how to prevent them. **Why:** Security is a compliance and trust requirement. RLS and pgaudit are increasingly tested in senior DB interviews. **Outputs:** Can implement a secure PostgreSQL deployment. Can write RLS policies. Can audit database access.

15 — Backup and Recovery

What: `pg_dump` (plain, custom, directory, tar formats), `pg_dumpall`, `pg_restore`, `pg_basebackup`, WAL archiving, Point-In-Time Recovery (PITR) — setup, execution, and validation, continuous archiving with pgBackRest (stanza creation, full/incremental/differential backups, restore, parallel operations), backup verification, RTO/RPO planning, disaster recovery drills. **Why:** Every production database needs a tested backup and recovery strategy. PITR is a mandatory senior DBA skill. **Outputs:** Can implement and test a complete backup/recovery strategy including PITR.

16 — PostgreSQL for Data Engineering

What: Table partitioning (range, list, hash — design, maintenance, partition pruning), FDW-based federated queries, `COPY` for bulk loading, `UNLOGGED` tables, temporary tables, staging patterns, incremental loading with `INSERT ... ON CONFLICT`, logical replication as a CDC mechanism, PostgreSQL as a data warehouse

(columnar compression, parallel query), integration with dbt, integration with Apache Airflow. **Why:** PostgreSQL is increasingly used as the analytical backend for mid-size data platforms. **Outputs:** Can build a PostgreSQL-based data platform. Can implement CDC with logical replication. Can design partitioned tables for analytics.

17 — PostgreSQL for Backend Engineers

What: Connection management (why connections are expensive in PostgreSQL, PgBouncer in transaction mode, connection limits), schema migration strategies (Flyway, Liquibase, Alembic), zero-downtime migrations (adding columns, adding indexes concurrently, dropping columns safely), `LISTEN / NOTIFY` for async messaging, `pg_advisory_lock` for distributed locking, multi-tenancy patterns (shared schema, schema-per-tenant, database-per-tenant), ORM integration pitfalls, N+1 detection. **Why:** Backend engineers interface with PostgreSQL daily. This chapter prevents the most common production mistakes. **Outputs:** Can implement zero-downtime migrations. Can design connection pooling strategy. Can use LISTEN/NOTIFY for lightweight pub/sub.

18 — Architecture and Case Studies

What: Real-world system designs using PostgreSQL — ride-sharing platform (Uber-style), social media feed (Twitter-style), e-commerce platform, SaaS multi-tenant application, financial ledger system, audit log system, real-time analytics dashboard. Each case study covers schema design, indexing strategy, query patterns, scaling approach, and lessons learned. **Why:** System design interviews require you to reason about databases in context. These case studies give you concrete patterns to reference. **Outputs:** Can design PostgreSQL-backed systems at scale. Can discuss trade-offs in system design interviews.

19 — Interview Questions

What: Curated question bank organized by difficulty (Junior, Mid, Senior, Staff) and topic. Every question includes: the question, what the interviewer is really testing, a model answer, follow-up questions to expect, and common mistakes candidates make. **Why:** Interview questions are a genre. The format, depth, and vocabulary expected varies by level. This chapter teaches you the genre. **Outputs:** Ready for technical SQL and PostgreSQL interviews at any level.

20 — Interview Tasks

What: Hands-on interview-style exercises — live coding problems, schema design challenges, query optimization tasks, debugging broken queries, system design mini-problems. All problems include: problem statement, hints, solution, and explanation. **Why:** The most effective interview preparation is simulated interviews. These tasks are designed to replicate the experience. **Outputs:** Practiced in solving interview problems under time pressure.

21 — System Design

What: Database-focused system design framework. How to approach DB design questions, how to size hardware, how to reason about read/write ratios, how to choose between PostgreSQL and other databases, when to use read replicas, when to shard, how to handle hot partitions, how to design for compliance (GDPR, SOC2). **Why:** System design interviews are required for senior+ roles. Database knowledge is a major component. **Outputs:** Can lead a system design interview discussion around any database-heavy component.

22 — Projects

What: Six standalone portfolio projects — (1) E-commerce database with full analytics, (2) Real-time event tracking system, (3) Multi-tenant SaaS platform, (4) Financial transaction ledger, (5) Full-text search engine

using PostgreSQL, (6) Replication lab with Patroni. **Why:** Projects demonstrate applied skill in a way that question-and-answer cannot. **Outputs:** Portfolio of PostgreSQL projects to show in interviews.

23 — Company Preparation

What: Company-specific preparation for Amazon, Google, Microsoft, Meta, Uber, Netflix, Shopify, Stripe. Covers: known interview format, topics they emphasize, LC-style SQL questions they've asked, system design patterns they use (with PostgreSQL implications), and cultural fit aspects. **Why:** Each company has a distinct interview style. Targeted preparation improves pass rates significantly. **Outputs:** Ready for database rounds at specific target companies.

24 — Cheat Sheets

What: Quick-reference cards for: SQL syntax, PostgreSQL data types, `EXPLAIN` node types, index selection guide, isolation levels, lock compatibility matrix, `pg_stat_*` views reference, `postgresql.conf` tuning guide, psql meta-commands, pgBackRest commands, Patroni commands. **Why:** During interviews and production incidents, fast access to correct syntax matters. **Outputs:** Reference materials for ongoing use after completing the course.

25 — Capstone Program

What: End-to-end capstone challenge that integrates all 24 prior chapters. Build a production-grade, multi-tenant SaaS database platform from scratch: design the schema, implement security, set up monitoring, configure replication, implement backup/recovery, optimize queries, and document the system. **Why:** The capstone synthesizes all knowledge into a single demonstrable artifact. **Outputs:** A complete, production-grade PostgreSQL system as a portfolio centerpiece.

---

11. Projects Overview

#	Project	Chapters Applied	Difficulty	Est. Hours
1	E-Commerce Analytics Database	01–08	Intermediate	12–16h
2	Real-Time Event Tracking System	09–11, 16	Intermediate	10–14h
3	Multi-Tenant SaaS Platform	06, 14, 17	Advanced	16–20h
4	Financial Transaction Ledger	09, 14, 18	Advanced	12–16h
5	PostgreSQL Full-Text Search Engine	05, 07, 11	Intermediate	8–12h
6	HA Replication Lab	10, 12, 13, 15	Expert	20–24h
7	Capstone: Production SaaS DB	All chapters	Expert	40–60h

---

12. Interview Readiness Tracker

Use this tracker to assess your readiness before applying.

Skill Area	Topics Covered	Self-Rating (1–5)
Basic SQL	SELECT, JOIN, GROUP BY, HAVING	__

Intermediate SQL	Subqueries, CTEs, Window Functions	__
Advanced SQL	Recursive CTEs, Lateral, Pivots	__
Schema Design	Normalization, Constraints, ERD	__
Indexes	B-Tree, GIN, partial, expression	__
Query Plans	EXPLAIN ANALYZE, plan nodes	__
Transactions	ACID, MVCC, isolation levels	__
Locking	Row locks, deadlocks, FOR UPDATE	__
PostgreSQL Internals	WAL, TOAST, heap, pages	__
Replication	Streaming, logical, Patroni	__
Backup/Recovery	PITR, pgBackRest	__
Security	RLS, pgaudit, privileges	__
Performance Tuning	autovacuum, checkpoint, config	__
System Design	HA, sharding, partitioning	__

Readiness Levels:

- Average 1–2: Complete Beginner Path
- Average 2–3: Backend/Analytics Path
- Average 3–4: Senior Engineering topics (10–15)
- Average 4–5: Staff level — ready to apply

---

13. Community and Contributing

How to Contribute

We welcome contributions from the community. Here's how:

1. **Fix a bug or typo** — Open a PR with the fix and a brief description.
2. **Add an example** — Add a `.sql` file with a well-commented new example to the relevant chapter folder.
3. **Add an interview question** — Add to the relevant section of `19_Interview_Questions` following the existing format.
4. **Add a project** — New projects should include: problem statement, schema, sample data, exercises, and solution.
5. **Improve an explanation** — If an explanation is confusing, rewrite it and open a PR.

Contribution Standards

- All SQL must be tested against PostgreSQL 16
- Examples must include expected output as SQL comments
- Explanations must be technically accurate — cite sources when making specific claims about internals
- No AI-generated filler content — every sentence must add value
- Follow the existing formatting conventions

Reporting Issues

Open a GitHub issue for:

- Technical inaccuracies
- Broken SQL examples
- Missing topics that should be covered
- Typos and grammatical errors

---

14. Acknowledgements

This repository draws on the following exceptional resources, which are recommended as supplementary reading:

Books

- *PostgreSQL: Up and Running* — Regina Obe, Leo Hsu
- *The Art of PostgreSQL* — Dimitri Fontaine
- *PostgreSQL High Performance* — Gregory Smith, Matthew Aiken, Ibrar Ahmed
- *Database Internals* — Alex Petrov
- *Designing Data-Intensive Applications* — Martin Kleppmann

Documentation

- PostgreSQL Official Documentation — The authoritative reference
- The Internals of PostgreSQL — Hironobu Suzuki

Blogs and Resources

- Cybertec PostgreSQL Blog (https://www.cybertec-postgresql.com/en/blog/)
- 2ndQuadrant / EDB Technical Blog
- Depesz Blog (https://www.depesz.com/) — EXPLAIN EXPLAIN
- pganalyze Blog (https://pganalyze.com/blog)
- Percona PostgreSQL Blog

---

15. License

"The difference between a good engineer and a great one is understanding what happens when you're not looking."

Start with `01_Fundamentals` if you're new. Start with `LEARNING_PATH.md` if you know where you're going.